



Build Brilliant

A Learn-by-Doing App, Built Deep for One Subject

Model the product on Brilliant, but pick a single subject and go deep instead of wide.

Background

Most learning apps hand you a video and a quiz. Brilliant does the opposite. You learn by doing. Every lesson drops you into a problem, lets you poke at it, gives instant feedback, and only then shows the idea behind it. You play with a concept until it clicks instead of watching someone explain it.

That is harder to build than a video player. It means real interactive problems, not multiple-choice trivia. It means visual simulations the learner can manipulate. It means instant, specific feedback on every tap. And it means a path that adapts: track what someone has mastered, fill gaps before they grow, and recommend what to do next. Wrap it in streaks and milestones so people come back tomorrow.

Your job: pick one subject and build a learn-by-doing app that teaches it well. Real interactions, real mastery tracking, a path that brings people back. Depth in one area, not a shallow tour of many.

Why This Matters

Passive content does not stick. Active problem-solving does. You are building the thing that makes hard ideas feel easy: a lesson someone can touch, get wrong, and figure out on their own. Pick a subject you can make genuinely great, and the depth will show.

Pick Your Domain

Before you write code, commit to one subject. The whole platform gets built for that subject. This is the most important decision you will make this week, so choose something you can teach

through interaction, not just text.

Good picks, with the kind of interaction that fits each:

- **Algebra:** drag terms across an equation, balance both sides, plot lines on a grid.
- **Probability and statistics:** run simulations, drag distributions, sample from a deck or dice and watch the result.
- **Physics:** adjust a slider on a projectile, a circuit, or a pendulum and watch it respond.
- **Python or programming:** small code or logic puzzles with instant run-and-check feedback.
- **Geometry:** drag points, measure angles, build and transform shapes.
- **Logic and puzzles:** deduction grids, truth tables, step-by-step reasoning you build.

You can pick something off this list or bring your own. The bar: it must be teachable through hands-on interaction and visuals, and you must be able to build several real lessons in it this week. State your chosen subject at the top of your README and design everything around it.

The Cadence: Build in Three Phases

This project has a strict order. Build the app first. Add intelligence second. Add learning science third. Do not skip ahead. Each phase has to stand on its own before the next one starts.

- **Phase 1, MVP by Wednesday:** the core learn-by-doing app. No AI. None.
- **Phase 2, AI features by Friday:** decide what AI should do here, then build it.
- **Phase 3, Learning science by Sunday:** layer evidence-based techniques on top.

The reason for the order: if the app does not teach without AI, no AI will save it. Prove the core experience works, then make it smarter, then make it stick.

Phase 1: MVP (by Wednesday)

This is a hard gate, and it has a hard rule: **no AI features in the MVP**. No model calls, no generated content, no chatbot tutor. Build the learn-by-doing app by hand so the core experience stands on its own. To pass, you need:

- A chosen subject, stated clearly, with the whole app built for a specific user persona
- One interactive lesson on a real concept in that subject, built around hands-on problems, not a video or a wall of text
- At least one problem the learner manipulates directly (drag, tap, adjust a slider, plot a point, reorder steps)
- A visual element they can interact with (a diagram, simulation, or chart that responds), appropriate to your subject
- Instant, specific feedback on each answer, right or wrong, with a short explanation, written by you, not generated
- Progress that persists: finish part of a lesson, come back, pick up where you left off
- Accounts and names (auth)
- Works on mobile screen sizes
- Deployed and public

The MVP is not about how many lessons you have. It is about whether one lesson actually teaches your subject without any AI doing the work. A single hand-built lesson that makes a hard idea click beats a library of click-next slideshows.

Example Architecture

At minimum:

1. A content model that describes a lesson as a sequence of interactive steps (concept, problem, feedback), not a blob of HTML. This is what lets you add lessons in your subject fast, and later lets AI generate them.
2. A frontend that renders those steps, captures the learner's interactions, and gives instant feedback driven by the content model.
3. A progress and mastery layer that records what the learner has done, what they got right, and what to show next within your subject.
4. A persistence layer so progress, streaks, and history survive across sessions and devices.

Interactive Lessons

A lesson is a short sequence of steps, each one interactive. A step introduces an idea, then makes the learner do something with it: solve a problem, manipulate a figure, predict an outcome, sort items. Every answer gets instant feedback. Wrong answers get a hint or an explanation, not just a red X. Keep lessons short, a few minutes each, so finishing one feels good.

Build at least one rich problem type beyond multiple choice, and make it fit your subject. A slider that changes a graph, a draggable diagram, a number line you plot on, a tap-to-build sequence, a small code or logic puzzle. The interaction should teach the idea, not decorate it.

Visual and Hands-On

Brilliant lessons are visual. A concept should have something the learner can see change as they act. Show a simulation, animate a result, or update a chart in response to input. The learner should be able to experiment and watch what happens. The right visual depends on your subject, so design it for the concepts you are teaching.

Course Path and Mastery

Group your lessons into a course with a clear path through the subject. Track what the learner has mastered and unlock or recommend what comes next. Remember where they stopped. When they get something wrong repeatedly, surface a review or an easier step before moving on. The path should feel like it knows where they are in your subject.

The Habit Loop

People learn when they come back. Add streaks, milestones, and a sense of daily progress. Show how far they have come and what is next. Make finishing a lesson satisfying. This is not

decoration. It is the difference between an app people open once and one they open every day.

Persistence and Mobile

Progress, streaks, and history persist across sessions and devices. Leave mid-lesson, come back on your phone, and continue. The app works well on mobile screen sizes, since that is where a lot of learning happens.

MVP Testing Scenario

We will test with:

1. A learner completing one lesson in your subject end to end, getting some problems wrong, and using the feedback to recover.
2. A learner manipulating the interactive element and watching the visual respond in real time.
3. A learner leaving mid-lesson and returning to confirm progress and streak persist.
4. A learner finishing a lesson and seeing the path recommends a sensible next step in the subject.
5. The whole thing on a phone-sized screen.

MVP Performance Targets

- Feedback on an answer appears instantly, under 100ms.
- Interactive visuals stay smooth at 60 FPS while the learner manipulates them.
- Lessons load fast, under 2 seconds to first interaction.
- Works on mobile screen sizes with touch input.
- Multiple concurrent learners with no slowdown.

We test on your deployed app. Make it hold up under load.

Depth Over Breadth

This is the rule that decides Phase 1. We would rather see five excellent lessons that build on each other in one subject than thirty thin lessons spread across many. Pick a subject, map a real learning path through it, and make each lesson teach. A learner should be able to start your course knowing little and come out understanding something real. Breadth is easy and forgettable. Depth is what makes Brilliant Brilliant.

Phase 2: AI Features (by Friday)

Only start this once the MVP teaches well on its own. Phase 2 has two parts: decide, then build. **Decide.** Look at your app and your subject and pick the AI features that genuinely help. Do not bolt on a chatbot because everyone has one. Write down what you chose and why.

Good candidates:

- Generate new practice problems in your subject at the right difficulty, so the course never runs dry.
- Give a targeted hint when a learner is stuck, without handing over the answer.

- Adapt the path: pick the next lesson based on what the learner struggles with.
- Explain a wrong answer in plain language, tuned to what the learner actually did.

Build. Implement the features you chose. Ground every AI feature in the lesson's structured state, not raw text. Verify anything checkable against the subject's logic, and for math use an engine like SymPy or math.js so the AI never gives a wrong answer. The MVP must keep working with AI turned off, so build these as additions, not replacements.

Document your decision in the Brainlift: what you considered, what you shipped, and what you deliberately left out.

Phase 3: Learning Science (by Sunday)

Now make the learning actually stick. Phase 3 layers evidence-based techniques on top of the working app. Pick the ones that fit your subject and implement them for real, not as buzzwords.

- **Retrieval practice:** make learners recall, not just recognize. Favor problems that pull the idea out of memory over passive review.
- **Spaced repetition:** bring concepts back at growing intervals, and resurface ones a learner got wrong sooner.
- **Interleaving:** mix problem types within a session instead of blocking them, so learners choose the right approach, not just the last one.
- **Mastery learning:** require real mastery of a concept before unlocking the next, with a clear mastery signal.
- **Scaffolding and desirable difficulty:** start supported, then fade the support so problems stay hard enough to grow from.
- **Immediate, explanatory feedback:** you have this from Phase 1. Sharpen it so wrong answers teach.

Pick a few, do them well, and measure or show their effect. In the Brainlift, explain which principles you used, how you implemented them, and why they fit your subject.

Build Strategy

Start with One Great Lesson

The interactive lesson is the hard part, so build it first, by hand, no AI. Pick one concept in your subject. Make a lesson that teaches it through a hands-on problem with a visual the learner can manipulate and instant feedback. Get that one lesson genuinely good before you build anything else.

Build Vertically, in Order

Finish one layer at a time, and respect the phases:

1. One interactive lesson in your subject, driven by a content model (Phase 1)
2. Instant feedback, including helpful responses to wrong answers (Phase 1)
3. Progress and persistence across sessions (Phase 1)
4. A course path with mastery tracking and a next-step recommendation (Phase 1)
5. Streaks and milestones (Phase 1)

6. Chosen AI features, grounded in lesson state (Phase 2)

7. Learning science techniques layered on top (Phase 3)

No half-finished features, and no skipping ahead. One lesson that truly teaches beats a pile of shallow ones, and an MVP that works without AI beats a fragile app that needs it.

Test Continuously

Test like we will. Get problems wrong on purpose and see if the feedback helps. Leave mid-lesson and come back. Open it on a phone. Turn AI off and confirm the app still teaches. Hand it to someone who does not know your subject and watch whether they learn it. That last test matters most.

Submission Requirements

Due **Sunday 10:59 PM CT**:

1. **GitHub repo**, with your chosen subject stated up front, a setup guide, architecture overview, and deployed link.
2. **Demo video (3 to 5 minutes)**, showing a full lesson in your subject, the interactive and visual elements, instant feedback, the course path and progress, your AI features, the learning science you added, and your architecture.
3. **Brainlift (1 page)**.
4. **Deployed app**, public, with auth, working on mobile, supporting multiple concurrent learners, and still teaching with AI turned off.

Deploy on Vercel, Firebase Hosting, or Render for simplicity.

Brainlift

Submit a 1-page brainlift documenting your AI-first build process and the point of view you developed. Cover: (1) Tools and workflow, what AI coding tools you used and how; (2) Prompting strategies, 3 to 5 prompts that worked, including how you got AI to help build interactive lessons in your subject; (3) Phase decisions, which AI features you chose and skipped in Phase 2, and which learning science principles you added in Phase 3 and why; (4) Code analysis, rough split of AI-generated vs hand-written; (5) Key learnings, the spiky opinions and insights you walked away with.

Technical Stack

Recommended, not required:

- Backend: Firebase, Supabase, or a custom server for auth, progress, and content.
- Frontend: React, Vue, or Svelte. HTML5 Canvas, SVG, or a library like D3 or Konva.js for interactive visuals and simulations.
- Content: a structured lesson format (JSON or similar) so you can add lessons in your subject without rewriting code, and so AI can generate them later.
- Phase 2 AI: OpenAI or Anthropic Claude for problem generation and hints, with a math engine (SymPy or math.js) for ground-truth checks where relevant.

Use whatever stack ships the best product.

Final Note

This is what the best learning companies are building right now: not videos, but experiences you learn by doing. Build the app first, make it smart second, make it stick third. Pick one subject, go deep, and build a lesson someone can touch, get wrong, and figure out on their own. The closer you get, the more you understand what it takes to put a world-class teacher in every pocket.

One subject taught deeply, by an app that works before the AI is even turned on, beats a feature-rich app that covers everything and teaches nothing.

Build something real.